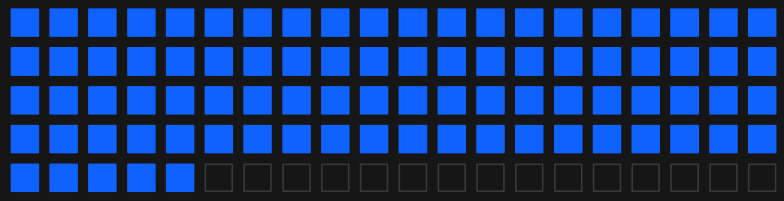
QMS1 · ACME.APPDATA.QUEUE

MQ AI Agent Ticket Pipeline

From a silent queue to an actionable diagnosis



850 messages waiting · 0 applications reading

A.J. Aronoff · IBM AngelHack · October 2026

Overview: What we are building

An automated pipeline that records status, notices problems, writes the ticket, enhances the ticket for the MQ AI Agent and asks the agent what is wrong.

DETECT

Sweep every queue. Depth with nothing reading it is the state that always needs a person.

```
CURDEPTH > 0
IPPROCS = 0
```

→

TICKET

Each stuck queue becomes one ticket, carrying the MQ facts that prove it.

```
INC-0042   QMS1
850 deep, 0 readers
```

→

ENHANCE

One line is added — the line that turns a record of a problem into a question an agent can answer.

```
Diagnose Q <queue>
on Q Manager <qmgr>
```

→

DIAGNOSE

The IBM MQ AI Agent reads it, inspects the live queue manager, and names the cause and the command.

```
GET(DISABLED)
found in 20 seconds
```

Nobody has to notice first (automated: detection, ticketing, enhancement)

Detect, ticket and enhance are ours, and they run unattended. **Diagnose is IBM's MQ AI Agent, live. The fix stays a human decision.**

A queue can be 85% full with no one reading from it

850 / 1000

ACME.APPDATA.QUEUE

85% full and climbing

3600 / 4000

ACME.COMMERCE.QUEUE

last put: a week ago

NONE

Connected user applications

nothing is draining either queue

IBM MQ already knew all of this.

Every number on this slide came off the QMS1 dashboard. The gap is not visibility — it is that nothing turns what MQ already reports into something a person is handed.

Source: IBM MQ Console, queue manager QMS1, 9 October 2026, 12:30 AM

Five stages, one instruction

01 lab

Break

Create QMS1, load the queue, set GET(DISABLED)

02 ours

Detect

Sweep every queue for depth with no reader

03 ours

Ticket

One ticket per stuck queue, with its MQ facts

04 ours

Enhance

Add the instruction that makes it answerable

05 IBM

Diagnose

IBM MQ Agent reads it and replies

AI

Stages 2 to 4 are ours. Stage 5 is the IBM MQ Agent, **running live in an IBM-provided sandbox — not a mock**, not a recording.

Building a queue that nobody can drain

```
crtmqm QMS1  &&  strmqm QMS1

runmqsc QMS1 <<EOF
  DEFINE QLOCAL('ACME.APPDATA.QUEUE') +
         MAXDEPTH(1000)
EOF

# load the backlog with IBM's own utility
dmpmqmsg -m QMS1 -f appdata.msgs +
         -o ACME.APPDATA.QUEUE

# now break it
runmqsc QMS1 <<EOF
  ALTER QLOCAL('ACME.APPDATA.QUEUE') +
        GET(DISABLED)
EOF
```

- **Load first, disable second**

  GET(DISABLED) blocks consumers, not producers — so the order is not strictly required. We do it anyway, because it mirrors how the fault really happens.

- **dmpmqmsg is IBM's own tool**

  The queue load and unload utility, formerly qload, shipped with MQ since version 8.0. Reading a file onto a queue is a plain PUT.

- **No consumer is ever started**

  That is the whole point. IPPROCS stays at zero, and the depth has nowhere to go.

One sweep, every queue

```
echo "DISPLAY QSTATUS(*) TYPE(QUEUE) \
      CURDEPTH IPPROCS OPPROCS" \
  | runmqsc QMS1
```

THE RULE

CURDEPTH > 0 AND IPPROCS = 0

Work is waiting, and no application has the queue open for input. Nothing can drain it.

WHERE IT RUNS

One MQSC call per queue manager, on a schedule. The MQ Agent covers up to twenty queue managers; the sweep covers the same estate in one pass.

SCANNER OUTPUT · QMS1

QUEUE	DEPTH	IPPROCS	
ACME.APPDATA.QUEUE	850	0	TICKET
ACME.COMMERCE.QUEUE	3600	0	TICKET
SYSTEM.DEFAULT.LOCAL.QUEUE	0	0	ok

Two tickets, not one. The sweep found a queue we had not broken on purpose — stuck since last week.

Depths are live from QMS1. No connected user applications — IBM MQ Console, Overview.

The enhancement is one line

TICKET AS RAISED

ID	INC-0042
SOURCE	qstatus sweep
QMGR	QMS1
QUEUE	ACME.APPDATA.QUEUE
CURDEPTH	850
IPPROCS	0
SEVERITY	High

Accurate, and useless to an agent.

→

AFTER THE ENHANCER

ID	INC-0042
QMGR	QMS1
QUEUE	ACME.APPDATA.QUEUE
CURDEPTH	850

INSTRUCTION

Diagnose Q ACME.APPDATA.QUEUE
on Q Manager QMS1

Two field substitutions. That is the whole enhancer.

Template: `Diagnose Q <queue from ticket> on Q Manager <queue manager from ticket>`

The answer

SEVERITY HIGH

PRIMARY ISSUE

The queue is GET-DISABLED, so applications cannot retrieve messages. This is why the depth is growing.

CURRENT METRICS

CURDEPTH 850 / MAXDEPTH 1000 **85% full**

UNCOM 0 no in-flight transactions MONQ off no put/get timestamps

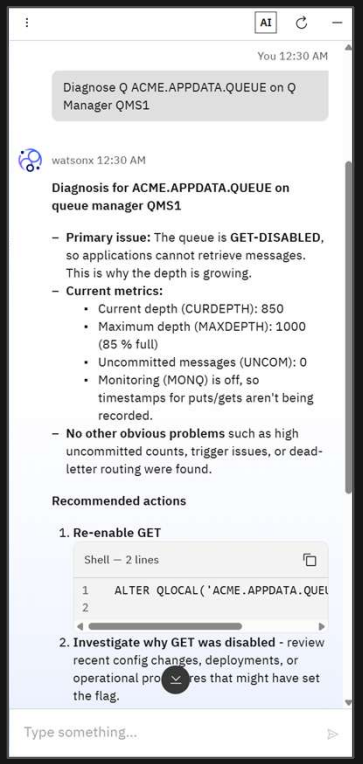
Ruled out: high uncommitted counts, trigger issues, dead-letter routing

RECOMMENDED ACTIONS

```
ALTER QLOCAL('ACME.APPDATA.QUEUE') GET(ENABLED)
```

Then: investigate why GET was disabled — recent config changes, deployments, or operational procedures.

Decomposed from the live reply. The unedited transcript is in the appendix.



as it appeared

AI

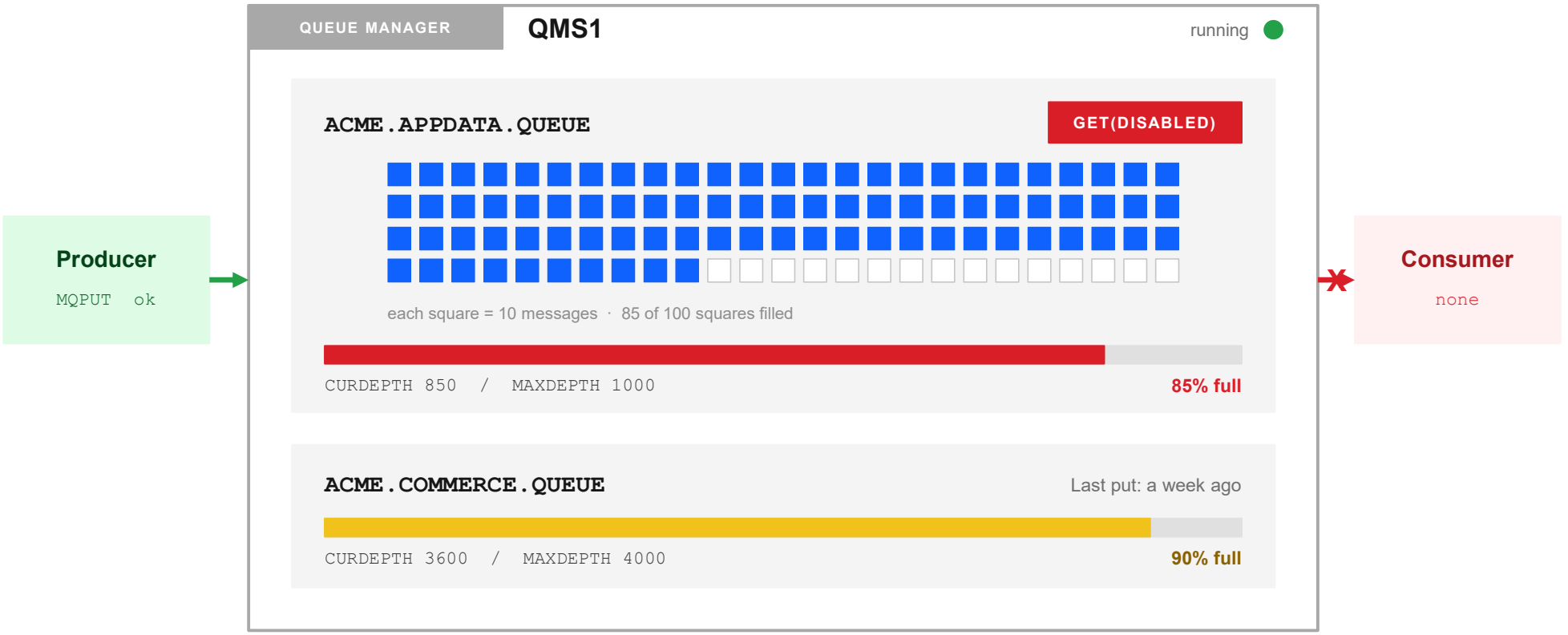
20 seconds

from the enhanced ticket to a named root cause and the exact command that fixes it.

Three other causes ruled out without being asked.

Nobody ran a single MQSC command.

QMS1, as the agent found it



No connected user applications — IBM MQ Console, Overview

What answers the ticket

MCP server

Hosts a suite of tools for gathering information from your IBM MQ queue managers.

Agent runtime

What you talk to. Uses AI models, and the tools hosted by the MCP server.
The agent Infers Intent, Analyzes Queries, Resolves Ambiguities

Up to 20 queue managers

Which can be running anywhere in your estate.

The three components of the IBM MQ Agent, in IBM's words

REQUIRES

MQ Advanced or Cloud Pak for Integration · Red Hat OpenShift · MQ Console 9.4.5 or later

AI

Run live, not mocked

IBM-provided sandbox, 9 Oct 2026

The agent cannot run MQSC commands to make changes. It reads your estate and tells you what is wrong.
Changing anything is still a decision somebody makes.

The agent recommends, people still decide

BEFORE

Someone notices

A consumer team complains, or a depth alert fires. An administrator opens the console, finds the queue, reads QSTATUS, then goes looking through recent changes for why GET is off.

NOW

The ticket arrives explaining itself

The sweep finds every stuck queue, raises a ticket per queue, and the enhancer adds one instruction. The agent answers with the cause and the command.

NEXT

The agent takes the ticket directly

This version makes the ticket easy to enhance. The next version of the agent can consume that enhanced ticket itself, with remediation behind an approval step.

The fix was always one command.

Everything we built exists to get a person to it faster.

```
ALTER QLOCAL('ACME.APPDATA.QUEUE') GET(ENABLED)
```

QMS1 · ACME.APPDATA.QUEUE · DIAGNOSED

Thank you

Questions?
During questions, I'll run a recreation of the Qs
And re-ask the MQ AI agent for a diagnosis

A.J. Aronoff · IBM AngelHack · October 2026

WHAT IS NEXT

- Depth-trend detection, to catch the slow consumer
- The agent consumes the enhanced ticket directly
- Remediation behind an approval step



GET(ENABLED) · depth 0

Why CURDEPTH and IPPROCS

`CURDEPTH > 0`

There is work waiting on the queue. On its own this is normal — a healthy queue has depth all the time.

`IPPROCS = 0`

No application has the queue open for input. On its own this is also normal — an idle queue has no readers.

`Both together`

Messages are arriving and nothing can take them. This is the state that always needs a human, and it is cheap to detect across an entire estate.

What this rule does not catch

A slow consumer

IPPROCS is above zero but depth still climbs. Needs a depth trend over time, not a single reading.

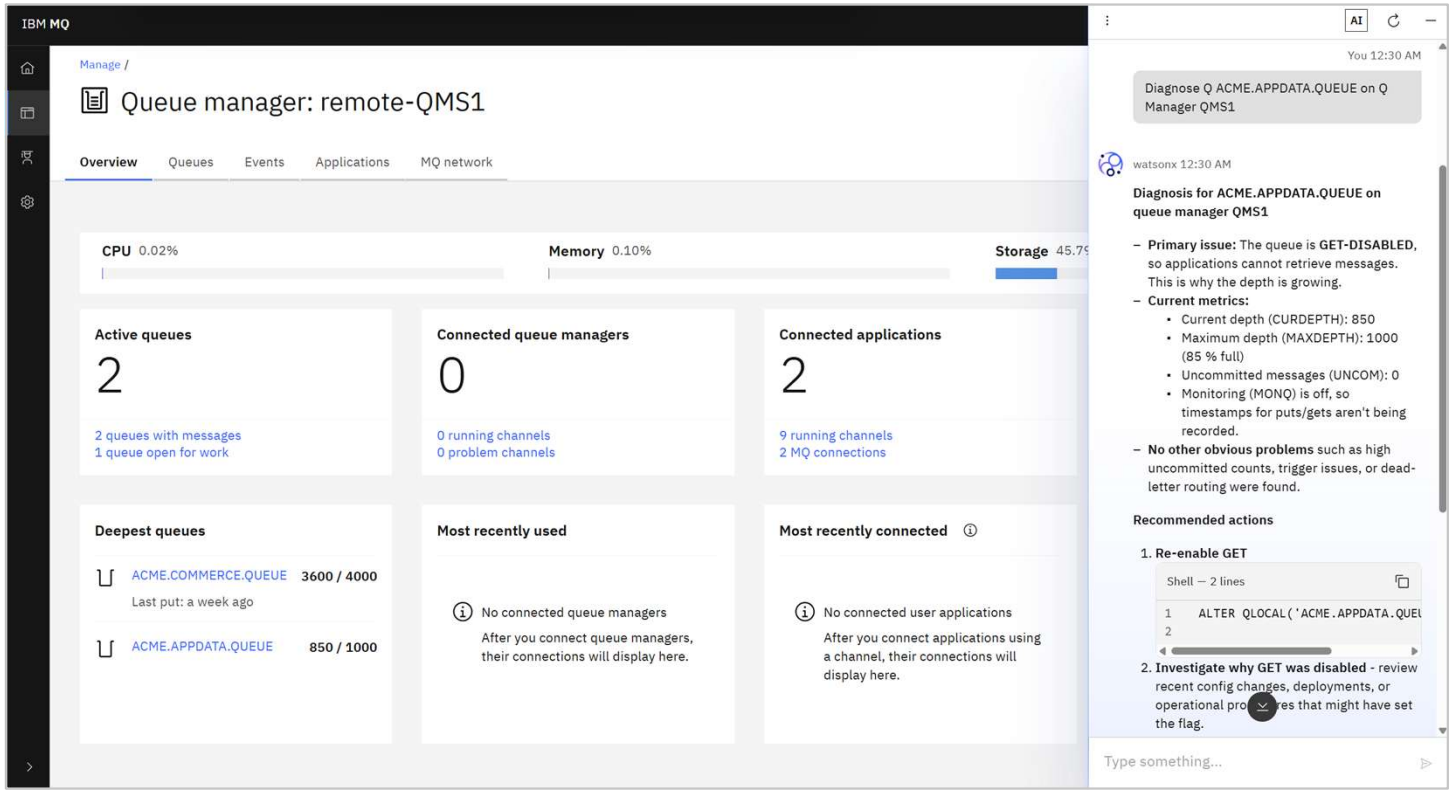
A queue that is meant to be deep

Some queues are buffers by design. Needs an allow-list, or a per-queue threshold.

Why MONQ matters

With monitoring off there are no put and get timestamps, so the agent could tell us what was wrong but not how long it had been wrong.

Evidence: QMS1, live



- **remote-QMS1**
  The console's name for the remote connection. The agent answers on queue manager QMS1.
- **850 / 1000**
  ACME.APPDATA.QUEUE, the queue in the ticket. ACME.COMMERCE.QUEUE sits above it at 3600 / 4000.
- **No connected user applications**
  The console's own words for IPPROCS = 0.
- **The AI button**
  Opens the agent panel. The reply is watsonx-branded.

Screenshot cropped to the application window. 9 October 2026, 12:33 AM.

The agent's reply, unedited

You 12:30 AM

Diagnose Q ACME.APPDATA.QUEUE on Q Manager QMS1

watsonx 12:30 AM

Diagnosis for ACME.APPDATA.QUEUE on queue manager QMS1

- Primary issue: The queue is GET-DISABLED, so applications cannot retrieve messages. This is why the depth is growing.
- Current metrics:
  - Current depth (CURDEPTH): 850
  - Maximum depth (MAXDEPTH): 1000 (85 % full)
  - Uncommitted messages (UNCOM): 0
  - Monitoring (MONQ) is off, so timestamps for puts/gets aren't being recorded.
- No other obvious problems such as high uncommitted counts, trigger issues, or dead-letter routing were found.

Recommended actions

1. **Re-enable GET**
   ALTER QLOCAL('ACME.APPDATA.QUEUE') GET(ENABLED)

2. **Investigate why GET was disabled**
   - review recent config changes, deployments, or operational procedures that might have set the flag.

3. **Optionally enable monitoring**
   ALTER QLOCAL('ACME.APPDATA.QUEUE') MONQ(ENABLED)

4. After enabling GET, monitor the depth trend and verify consuming applications are running correctly.

If the issue persists: check the consumer applications, trigger definitions, and any dead-letter or expiry handling affecting the queue.

Left column transcribed from the screenshot. Right column continues below the visible scroll, from the saved conversation.